

Module 3 Notes: Traversal, Display, and C List Creation

1. Current progress analysis

You have already covered the two foundation ideas needed for Module 3.

Module 1 foundation

You learned why linked lists exist.

Arrays are fast when we know the index, but arrays have a fixed size after creation. Linked lists are more flexible because they can grow and shrink one node at a time.

Main tradeoff:

Structure	Strength	Weakness
Array	Fast direct access by index	Fixed size and shifting during insertion/deletion
Linked list	Flexible size and pointer-based insertion/deletion	No direct jumping to an index

Module 2 foundation

You learned the building block of a linked list: the node.

A singly linked-list node has:

[data | next]

Meaning:

Part	Meaning
data	The actual value stored in the node
next	Address of the next node

You also learned the most important pointer movement:

```
p = p->next;
```

That moves pointer `p` forward by one node.

So your current level is this:

```
You understand what a node is.  
You understand what first/head means.  
You understand how one pointer moves to the next node.
```

Module 3 now combines those ideas into the first real linked-list operations:

```
Traversal + Display + Create
```

2. Big idea of Module 3

Module 3 answers two practical questions:

1. How do we visit every node in a linked list?
2. How do we create a full linked list from an array?

Example list:

```
first  
|  
v  
[3 | next] -> [5 | next] -> [7 | next] -> [10 | next] -> [15 | NULL]
```

The important process is:

```
Start at the first node.  
Read or print the current node.  
Move to the next node.  
Repeat until NULL.
```

This process is called traversal.

3. What is traversal?

Traversal means moving through a linked list node by node.

Example:

```
8 -> 7 -> 12 -> NULL
```

Traversal does this:

```
Visit 8  
Move to 7  
Visit 7  
Move to 12  
Visit 12  
Move to NULL  
Stop
```

The movement line is:

```
p = p->next;
```

This means:

```
Make p point to the next node.
```

4. Why we do not move `first` directly

Suppose the list is:

```
first  
|  
v  
[10] -> [20] -> [30] -> NULL
```

If we do this:

```
first = first->next;
```

then `first` moves to the second node.

Now the list looks like this from the program's point of view:

```
first
|
v
[20] -> [30] -> NULL
```

The node containing `10` may be lost if no other pointer stores its address.

So during traversal, we use a temporary pointer:

```
struct Node *p = first;
```

Now:

```
first stays at the beginning.
p moves through the list.
```

This is the safe traversal pattern.

5. Displaying a linked list

Display means printing every node's data.

For this list:

```
8 -> 7 -> 12 -> NULL
```

Display should print:

```
8 7 12
```

Display function in C

```
void display(struct Node *p) {  
    while (p != NULL) {  
        printf("%d ", p->data);  
        p = p->next;  
    }  
}
```

Meaning of each line

```
void display(struct Node *p)
```

The function receives a pointer to the first node. Usually, we call it like this:

```
display(first);
```

Inside the function, `p` is a copy of `first`. Moving `p` does not move `first`.

```
while (p != NULL)
```

Keep looping while `p` points to a real node.

```
printf("%d ", p->data);
```

Print the data stored in the current node.

```
p = p->next;
```

Move `p` to the next node.

6. Dry run of `display`

Given list:

```
first -> [8] -> [7] -> [12] -> NULL
```

Call:

```
display(first);
```

At the start:

```
p points to 8
```

Loop 1

```
p != NULL is true  
print 8  
p moves to 7
```

Loop 2

```
p != NULL is true  
print 7  
p moves to 12
```

Loop 3

```
p != NULL is true  
print 12  
p moves to NULL
```

Loop stops

```
p != NULL is false
```

Final output:

```
8 7 12
```

7. Creating a linked list from an array

Suppose we have this array:

```
int A[] = {3, 5, 7, 10, 15};
```

We want to create this linked list:

```
3 -> 5 -> 7 -> 10 -> 15 -> NULL
```

To build the list, we use three important pointers.

Pointer	Role
<code>first</code>	Points to the first node of the list
<code>last</code>	Points to the current last node
<code>t</code>	Temporary pointer used for the new node being created

8. Why the first node is special

At the beginning, the list is empty:

```
first = NULL
```

There is no first node. There is no last node.

So the first node must be created separately.

```
first = (struct Node *)malloc(sizeof(struct Node));
first->data = A[0];
first->next = NULL;
last = first;
```

After this:

```
first
|
v
[3 | NULL]
^
```

```
|  
last
```

`first` and `last` both point to the same node because there is only one node.

9. Creating the remaining nodes

After the first node is created, each new node follows the same pattern.

For every next array value:

```
Create a new node t.  
Put the array value into t->data.  
Set t->next to NULL.  
Connect old last node to t.  
Move last to t.
```

In C:

```
t = (struct Node *)malloc(sizeof(struct Node));  
t->data = A[i];  
t->next = NULL;  
  
last->next = t;  
last = t;
```

The most important line is:

```
last->next = t;
```

This attaches the new node to the existing chain.

Then:

```
last = t;
```

This updates `last` so it points to the new final node.

10. Full Module 3 C program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *first = NULL;

void create(int A[], int n) {
    struct Node *t;
    struct Node *last;

    first = (struct Node *)malloc(sizeof(struct Node));
    first->data = A[0];
    first->next = NULL;

    last = first;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = NULL;

        last->next = t;
        last = t;
    }
}

void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}

int main() {
    int A[] = {3, 5, 7, 10, 15};

    create(A, 5);
    display(first);
}
```

```
    return 0;  
}
```

Output:

```
3 5 7 10 15
```

11. Dry run of `create(A, 5)`

Array:

```
A = {3, 5, 7, 10, 15}
```

Step 1: Create first node with 3

```
first  
|  
v  
[3 | NULL]  
^  
|  
last
```

Step 2: Add 5

Create `t`:

```
[5 | NULL]  
^  
|  
t
```

Connect it:

```
last->next = t;
```

Now:

```
first
|
v
[3] -> [5] -> NULL
      ^
      |
      last
```

Step 3: Add 7

```
first
|
v
[3] -> [5] -> [7] -> NULL
              ^
              |
              last
```

Step 4: Add 10

```
first
|
v
[3] -> [5] -> [7] -> [10] -> NULL
                      ^
                      |
                      last
```

Step 5: Add 15

```
first
|
v
[3] -> [5] -> [7] -> [10] -> [15] -> NULL
                                ^
                                |
                                last
```

Final list:

```
first -> 3 -> 5 -> 7 -> 10 -> 15 -> NULL
```

12. Why `t->next = NULL` matters

Every new node is inserted at the end.

The last node of a normal singly linked list must point to `NULL`.

So when creating a new node:

```
t->next = NULL;
```

This says:

```
This new node is currently the last node.
```

Later, if another node is added, the old last node's `next` will be updated.

13. Why `last = t` matters

After connecting the new node:

```
last->next = t;
```

we must move `last` forward:

```
last = t;
```

If we forget this, `last` keeps pointing to the old node, and future insertions may overwrite links or build the list incorrectly.

Correct pattern:

```
last->next = t;  
last = t;
```

Think of `last` as a bookmark that must always stay on the final node.

14. Complexity of Module 3 operations

Display complexity

Display visits each node once.

If there are n nodes:

```
Time:  $O(n)$   
Extra space:  $O(1)$ 
```

Why $O(n)$?

Because this loop runs once per node:

```
while (p != NULL) {  
    printf("%d ", p->data);  
    p = p->next;  
}
```

Why $O(1)$ extra space?

Because display uses only one temporary pointer, p .

Create complexity

Create makes one node for every array element.

If the array has n elements:

```
Time:  $O(n)$   
Space:  $O(n)$ 
```

Why $O(n)$ time?

Because we create and link n nodes.

Why $O(n)$ space?

Because the linked list stores n nodes in heap memory.

15. Beginner mistakes to avoid

Mistake 1: Moving `first` during display

Wrong:

```
while (first != NULL) {  
    printf("%d ", first->data);  
    first = first->next;  
}
```

This loses the original start of the list.

Correct:

```
void display(struct Node *p) {  
    while (p != NULL) {  
        printf("%d ", p->data);  
        p = p->next;  
    }  
}
```

Mistake 2: Forgetting `t->next = NULL`

Wrong:

```
t->data = A[i];  
last->next = t;  
last = t;
```

Correct:

```
t->data = A[i];  
t->next = NULL;  
last->next = t;  
last = t;
```

Mistake 3: Forgetting `last = t`

Wrong:

```
last->next = t;
```

Correct:

```
last->next = t;  
last = t;
```

Mistake 4: Forgetting `#include <stdlib.h>`

Since `malloc` is used in C, include:

```
#include <stdlib.h>
```

Mistake 5: Accessing `p->data` after `p` becomes `NULL`

Wrong:

```
while (p != NULL) {  
    p = p->next;  
    printf("%d ", p->data);  
}
```

This can crash when `p` becomes `NULL`.

Correct:

```
while (p != NULL) {  
    printf("%d ", p->data);  
    p = p->next;  
}
```

Print first, then move.

16. Debug display version

A debug display can help you see pointer movement.

```
void debugDisplay(struct Node *p) {  
    while (p != NULL) {
```

```

        printf("Current node data: %d\n", p->data);
        printf("Current node address: %p\n", (void *)p);
        printf("Next node address: %p\n\n", (void *)p->next);
        p = p->next;
    }
}

```

This prints each node's data, its own address, and the address stored in `next`.

The exact addresses will be different every time the program runs.

17. Safer version of `create`

The basic version assumes `n > 0`.

A safer version checks for invalid size:

```

void create(int A[], int n) {
    if (n <= 0) {
        first = NULL;
        return;
    }

    struct Node *t;
    struct Node *last;

    first = (struct Node *)malloc(sizeof(struct Node));
    first->data = A[0];
    first->next = NULL;

    last = first;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = NULL;

        last->next = t;
        last = t;
    }
}

```

This prevents reading `A[0]` when there are no elements.

18. Module 3 final summary

Module 3 teaches the first practical linked-list operations.

Key ideas:

Traversal means visiting nodes one by one.
Display means traversing and printing data.
Create means building nodes and linking them together.

Key pointers:

```
first = start of the list
p      = temporary pointer used for traversal
last   = current end of the list
t      = temporary new node
```

Key code patterns:

```
p = p->next;
```

Moves through the list.

```
last->next = t;
last = t;
```

Adds a new node to the end of the list.

Most important rule:

Do not move first during traversal. Move a temporary pointer instead.

19. Practice tasks

Task 1

Write `display()` from memory.

Expected function:

```
void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}
```

Task 2

Dry run this list:

```
2 -> 4 -> 6 -> 8 -> NULL
```

Write what `display()` prints and where `p` points after each loop.

Task 3

Create a linked list from this array:

```
int A[] = {2, 4, 6, 8};
```

Final list should be:

```
2 -> 4 -> 6 -> 8 -> NULL
```

Task 4

Explain these pointers in your own words:

```
first
last
t
p
```

Task 5

Answer this:

Why is this line dangerous during display?

```
first = first->next;
```

Correct answer:

Because it moves the main pointer away from the first node, so the start of the list may be lost.